



暨南大學
JINAN UNIVERSITY

Undergraduate Lab Report

Course Title: Experiment of Computer Organization

Course No: 60080014

Student Name: 李榮耀

Student No: 2024103975

School: International School

Department: _____

Major: CST

Instructor: SUN Heng

Academic Year: 2025~2026, Semester : 1st [☒] 2nd [☐]

Academic Affairs Office of Jinan University

Date (dd/mm/yyyy) 2025/10/15

Computer Organization Lab List

Student Name: Li Rongyao Student No:

2024103975

ID	Lab Name	Type
1	Number Storage Lab	Individual
2	Manipulating Bits	Individual
3	A Simple Real-life Control System Lab	Team
4	System I/O	Individual
5	Simulating Y86-64 Program	Individual
6	Performance Lab	Team

Undergraduate Lab Report of Jinan University

Course Title Experiment of computer organization

Evaluation

Lab Name Number Storage Lab

Instructor Sun Heng

Lab Address Room 116

Student Name Li Rongyao

Student No 2024103975

College International School

Department Major CST

Date 2025/10/15 Afternoon

1. Introduction

This lab extends the section *Information Storage* (chapter 2) in the textbook. It will immerse us in a problem context, which will require us to apply the theoretical concepts taught in the C language course. This will help us to develop basic skills needed to become a device level system developers and digital system designer. While going through this lab concentrate on developing a good understanding of the problem context, as it is a fundamental requirement for becoming a skilled system designer or application developer.

2. Lab Instructions or Steps

a) Using less than 4 sentences, explain what function/action, the two statements marked with “Tag 1” and “Tag 2” perform.

b) Compile and run this program with “gcc” on the Linux machine.

c) When you ran your code, did you get the time executed to be negative? If yes, why did that happen? (Since time cannot be negative). How could you fix this? Figure out how to fix it and do the necessary modifications.

d) Change the data type of the variable `time_stamp` from *double* to *long int*. Is there a change in the values reported? If so, which of the values is the correct value? Why is there a difference?

e) Find out the structure of type “timeval”. Is it a standard C data type or is it platform specific?

f) Print the output files for all runs in the report. (and the fixed versions if necessary).

g) Write a C program to get bit and byte lengths for all the following C numerical data types according to the following table. The new C code file should have name “lab1_2.c” and it should generate an output file named “lab1_2_out.txt”.

3. Lab Device or Environment

Ubuntu 16.04 (64-bit) on virtual machine (VMware workstation 17 Pro)

4. Results and Analysis

a) Tag 1 is to create a variable named *int_var* with the data type *int*.

And Tag 2 is to output the data size of *int* data type in bytes and bits respectively through the variable *int_var*.

b)

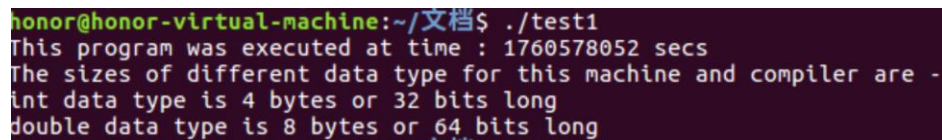
```
honor@honor-virtual-machine:~/文档$ gcc lab1.c -o lab1
lab1.c: In function 'main':
lab1.c:33:3: warning: implicit declaration of function 'exit' [-Wimplicit-functi
on-declaration]
    exit(1);
    ^
lab1.c:33:3: warning: incompatible implicit declaration of built-in function 'ex
it'
lab1.c:33:3: note: include '<stdlib.h>' or provide a declaration of 'exit'
lab1.c:43:27: warning: format '%d' expects argument of type 'int', but argument
3 has type 'long unsigned int' [-Wformat=]
    fprintf(my_file_pointer, "int data type is %d bytes or %d bits long\n",sizeof(
    ^
lab1.c:43:27: warning: format '%d' expects argument of type 'int', but argument
4 has type 'long unsigned int' [-Wformat=]
lab1.c:44:30: warning: format '%d' expects argument of type 'int', but argument
3 has type 'long unsigned int' [-Wformat=]
    fprintf(my_file_pointer, "double data type is %d bytes or %d bits long\n",s
    ^
lab1.c:44:30: warning: format '%d' expects argument of type 'int', but argument
4 has type 'long unsigned int' [-Wformat=]
lab1.c:50:9: warning: format '%d' expects argument of type 'int', but argument 2
has type 'long unsigned int' [-Wformat=]
    printf("int data type is %d bytes or %d bits long\n",sizeof(int_var), sizeof(i
    ^
lab1.c:50:9: warning: format '%d' expects argument of type 'int', but argument 3
has type 'long unsigned int' [-Wformat=]
lab1.c:51:9: warning: format '%d' expects argument of type 'int', but argument 2
has type 'long unsigned int' [-Wformat=]
    printf("double data type is %d bytes or %d bits long\n",sizeof(double), sizeof
    ^
lab1.c:51:9: warning: format '%d' expects argument of type 'int', but argument 3
has type 'long unsigned int' [-Wformat=]
honor@honor-virtual-machine:~/文档$ ./lab1
This program was executed at time : 1760517115 secs
The sizes of different data type for this machine and compiler are -
int data type is 4 bytes or 32 bits long
double data type is 8 bytes or 64 bits long
honor@honor-virtual-machine:~/文档$
```

When compile, there exists warning since the function *exit()* in *stdlib.h* is declaration and the output format *%d* for the type *int* cannot match the data type *long unsigned int* of arguments. And we can know the data type *double* and *int* converse implicitly.

c) No. The time is positive. Because the data type ``double`` converse implicitly to ``long unsigned int``, which is type cast rather than truncation on bits in C language. And the type ``long unsigned int``, which is 8-byte long on the 64-bit machine, is enough to store the time from 1970/01/01 to now. Even if it is the type ``int``, its upper bound $2^{31}-1$ is still enough to store the time from 1970/01/01 to now.

If the output is negative, it may occur the truncation of lower 32 bits when outputting with the format `%d`. That will lead to an error negative print.

d) After modifying the data type of the variable ``time_stamp`` from `double` to `long int`, the result is as:



```
honor@honor-virtual-machine:~/文档$ ./test1
This program was executed at time : 1760578052 secs
The sizes of different data type for this machine and compiler are -
int data type is 4 bytes or 32 bits long
double data type is 8 bytes or 64 bits long
```

There are no explicit changes because before modifying the data type has been converted implicitly into `long unsigned int`, which even can store more numbers than `long int` for positive.

e) Find the `struct timeval` in the function `gettimeofday()`:

DESCRIPTION

The functions `gettimeofday()` and `settimeofday()` can get and set the time as well as a timezone. The `tv` argument is a `struct timeval` (as specified in `<sys/time.h>`):

```
struct timeval {
    time_t      tv_sec;        /* seconds */
    suseconds_t tv_usec;      /* microseconds */
};
```

and gives the number of seconds and microseconds since the Epoch (see `time(2)`). The `tz` argument is a `struct timezone`:

```
struct timezone {
    int tz_minuteswest; /* minutes west of Greenwich */
    int tz_dsttime;     /* type of DST correction */
};
```

It consists of two elements `tv_sec` and `tv_usec`, which are with the data type `time_t` and `suseconds_t` respectively.

Find the description of `time_t` and `suseconds_t` in `types.h`:

```
__STD_TYPE __ID_T_TYPE __id_t; /* General type for IDs. */
__STD_TYPE __TIME_T_TYPE __time_t; /* Seconds since the Epoch. */
__STD_TYPE __USECONDS_T_TYPE __useconds_t; /* Count of microseconds. */
__STD_TYPE __SUSECONDS_T_TYPE __suseconds_t; /* Signed count of microseconds. */
```

And in `typesizes.h`:

```
/usr/include/x86_64-linux-gnu/bits/typesizes.h:#define __TIME_T_TYPE __SYSCALL_SLONG_TYPE
/usr/include/x86_64-linux-gnu/bits/typesizes.h:#define __SUSECONDS_T_TYPE __SYSCALL_SLONG_TYPE

#define __SYSCALL_SLONG_TYPE __SLONGWORD_TYPE
```

And in `types.h` again:

```
#define __SLONGWORD_TYPE long int
```

So we can know the struct `timeval` has two elements with the same data type of `long int`.

And in the description of `time.h`:

```
honor@honor-virtual-machine:/usr/include/x86_64-linux-gnu/bits$ cat time.h
/* System-dependent timing definitions. Linux version.
Copyright (C) 1996-2016 Free Software Foundation, Inc.
This file is part of the GNU C Library.
```

So we can know it is platform specific.

- f) The output of `lab1.c` and its fixed version by modifying the data type of `time_stamp` from `double` to `long int`:


```
honor@honor-virtual-machine:~/文档$ ./lab1
This program was executed at time : 1760582498 secs
The sizes of different data type for this machine and compiler are -
int data type is 4 bytes or 32 bits long
double data type is 8 bytes or 64 bits long
honor@honor-virtual-machine:~/文档$ ./test1
This program was executed at time : 1760582501 secs
The sizes of different data type for this machine and compiler are -
int data type is 4 bytes or 32 bits long
double data type is 8 bytes or 64 bits long
```

g) For 32-bit machine and 64-bit machine:

```
honor@honor-virtual-machine:~/文档$ ./lab1_2_32bit
The sizes of different data type for this machine and compiler are -
char data type is 1 bytes or 8 bits long
unsigned char data type is 1 bytes or 8 bits long
short data type is 2 bytes or 16 bits long
unsigned short data type is 2 bytes or 16 bits long
int data type is 4 bytes or 32 bits long
unsigned int data type is 4 bytes or 32 bits long
long data type is 4 bytes or 32 bits long
unsigned long data type is 4 bytes or 32 bits long
int32_t data type is 4 bytes or 32 bits long
uint32_t data type is 4 bytes or 32 bits long
int64_t data type is 8 bytes or 64 bits long
uint64_t data type is 8 bytes or 64 bits long
char* data type is 4 bytes or 32 bits long
float data type is 4 bytes or 32 bits long
double data type is 8 bytes or 64 bits long
honor@honor-virtual-machine:~/文档$ ./lab1_2_64bit
The sizes of different data type for this machine and compiler are -
char data type is 1 bytes or 8 bits long
unsigned char data type is 1 bytes or 8 bits long
short data type is 2 bytes or 16 bits long
unsigned short data type is 2 bytes or 16 bits long
int data type is 4 bytes or 32 bits long
unsigned int data type is 4 bytes or 32 bits long
long data type is 8 bytes or 64 bits long
unsigned long data type is 8 bytes or 64 bits long
int32_t data type is 4 bytes or 32 bits long
uint32_t data type is 4 bytes or 32 bits long
int64_t data type is 8 bytes or 64 bits long
uint64_t data type is 8 bytes or 64 bits long
char* data type is 8 bytes or 64 bits long
float data type is 4 bytes or 32 bits long
double data type is 8 bytes or 64 bits long
```

5. Appendix (Program Code)

Lab1_2.c:

```
#include <stdio.h> //For input/output
#include <stdint.h> //For int32_t
#include <stdlib.h> //For exit() function

int main()
{

    FILE *my_file_pointer;
```



```

if ((my_file_pointer = fopen("lab1_2_out.txt", "w")) == NULL)
{
    printf("Error opening the file, so exiting\n");
    exit(1);
}

// Code segment for file I/O

fprintf(my_file_pointer, "The sizes of different data type for this
machine and compiler are -\n");
fprintf(my_file_pointer, "char data type is %d bytes or %d bits long\n",
sizeof(char), sizeof(char) * 8);
fprintf(my_file_pointer, "unsigned char data type is %d bytes or %d bits
long\n", sizeof(unsigned char), sizeof(unsigned char) * 8);
fprintf(my_file_pointer, "short data type is %d bytes or %d bits long\n",
sizeof(short), sizeof(short) * 8);
fprintf(my_file_pointer, "unsigned short data type is %d bytes or %d bits
long\n", sizeof(unsigned short), sizeof(unsigned short) * 8);
fprintf(my_file_pointer, "int data type is %d bytes or %d bits long\n",
sizeof(int), sizeof(int) * 8);
fprintf(my_file_pointer, "unsigned int data type is %d bytes or %d bits
long\n", sizeof(unsigned int), sizeof(unsigned int) * 8);
fprintf(my_file_pointer, "long data type is %d bytes or %d bits long\n",
sizeof(long), sizeof(long) * 8);
fprintf(my_file_pointer, "unsigned long data type is %d bytes or %d bits
long\n", sizeof(unsigned long), sizeof(unsigned long) * 8);
fprintf(my_file_pointer, "int32_t data type is %d bytes or %d bits
long\n", sizeof(int32_t), sizeof(int32_t) * 8);
fprintf(my_file_pointer, "uint32_t data type is %d bytes or %d bits
long\n", sizeof(uint32_t), sizeof(uint32_t) * 8);
fprintf(my_file_pointer, "int64_t data type is %d bytes or %d bits
long\n", sizeof(int64_t), sizeof(int64_t) * 8);
fprintf(my_file_pointer, "uint64_t data type is %d bytes or %d bits
long\n", sizeof(uint64_t), sizeof(uint64_t) * 8);
fprintf(my_file_pointer, "char* data type is %d bytes or %d bits long\n",
sizeof(char*), sizeof(char*) * 8);
fprintf(my_file_pointer, "float data type is %d bytes or %d bits long\n",
sizeof(float), sizeof(float) * 8);
fprintf(my_file_pointer, "double data type is %d bytes or %d bits long\n",
sizeof(double), sizeof(double) * 8);

// Code segment for console I/O, this can be used instead of the file
I/O

```

```

    printf("The sizes of different data type for this machine and compiler
are -\n");
    printf("char data type is %d bytes or %d bits long\n", sizeof(char),
sizeof(char) * 8);
    printf("unsigned char data type is %d bytes or %d bits long\n",
sizeof(unsigned char), sizeof(unsigned char) * 8);
    printf("short data type is %d bytes or %d bits long\n", sizeof(short),
sizeof(short) * 8);
    printf("unsigned short data type is %d bytes or %d bits long\n",
sizeof(unsigned short), sizeof(unsigned short) * 8);
    printf("int data type is %d bytes or %d bits long\n", sizeof(int),
sizeof(int) * 8);
    printf("unsigned int data type is %d bytes or %d bits long\n",
sizeof(unsigned int), sizeof(unsigned int) * 8);
    printf("long data type is %d bytes or %d bits long\n", sizeof(long),
sizeof(long) * 8);
    printf("unsigned long data type is %d bytes or %d bits long\n",
sizeof(unsigned long), sizeof(unsigned long) * 8);
    printf("int32_t data type is %d bytes or %d bits long\n", sizeof(int32_t),
sizeof(int32_t) * 8);
    printf("uint32_t data type is %d bytes or %d bits long\n",
sizeof(uint32_t), sizeof(uint32_t) * 8);
    printf("int64_t data type is %d bytes or %d bits long\n", sizeof(int64_t),
sizeof(int64_t) * 8);
    printf("uint64_t data type is %d bytes or %d bits long\n",
sizeof(uint64_t), sizeof(uint64_t) * 8);
    printf("char* data type is %d bytes or %d bits long\n", sizeof(char *),
sizeof(char *) * 8);
    printf("float data type is %d bytes or %d bits long\n", sizeof(float),
sizeof(float) * 8);
    printf("double data type is %d bytes or %d bits long\n", sizeof(double),
sizeof(double) * 8);

    fclose(my_file_pointer); // To close the output file, mandatory to
actually get an output !

    return 0;
}

```

Lab1.c(Fixed)

```

/*
Instructions -

```

To run this program - first compile to create an executable, then run the executable code.

Type the following in the Linux console/shell to compile and make an executable using the gcc compiler-

```
gcc lab1.c -o lab1
```

To run the executable named "lab1", type the following-

```
./lab1
```

```
*/
```

```
#include <stdio.h> //For input/output
#include <stdint.h> //For int32_t
#include <sys/time.h> //For gettimeofday() function

int main()
{
    int int_var; //Tag 1

    struct timeval this_instant;
    long int time_stamp;

    FILE *my_file_pointer;
    if ( (my_file_pointer = fopen("lab1_out.txt", "w")) == NULL)
    {
        printf("Error opening the file, so exiting\n");
        exit(1);
    }

    gettimeofday(&this_instant,0);
    time_stamp = this_instant.tv_sec;

    //Code segment for file I/O
    fprintf(my_file_pointer, "This program was executed at time : %d secs\n",
(int)time_stamp);

    fprintf(my_file_pointer, "The sizes of different data type for this
machine and compiler are -\n");
    fprintf(my_file_pointer, "int data type is %d bytes or %d bits
long\n",sizeof(int_var), sizeof(int_var)*8 );
```

```
    fprintf(my_file_pointer, "double data type is %d bytes or %d bits  
long\n", sizeof(double), sizeof(double)*8 );  
  
    //Code segment for console I/O, this can be used instead of the file I/O  
    printf("This program was executed at time : %d secs\n", (int)time_stamp);  
  
    printf("The sizes of different data type for this machine and compiler  
are -\n");  
    printf("int data type is %d bytes or %d bits long\n", sizeof(int_var),  
sizeof(int_var)*8 ); //Tag 2  
    printf("double data type is %d bytes or %d bits long\n", sizeof(double),  
sizeof(double)*8 );  
  
    fclose(my_file_pointer); //To close the output file, mandatory to  
actually get an output !  
  
    return 0;  
}
```